ChatGPT

사이킷런 보스턴집값으로 텐서플로우 모델제작, 성능평가, 추론까지

사이킷런의 보스턴 집값 데이터를 사용하여 텐서플로우 모델을 만들고 성능 평가 및 추론까지 진행하는 예제를 단계별로 설명드리겠습니다.

1. 데이터 로드 및 전처리

먼저, 사이킷런에서 제공하는 보스턴 집값 데이터셋을 로드합니다.

```python
import tensorflow as tf
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# 보스턴 집값 데이터셋 로드
boston = load_boston()
X = boston.data
y = boston.target

# 데이터 분할 (훈련 데이터와 테스트 데이터)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 데이터 표준화 (스케일링)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. 텐서플로우 모델 정의

모델을 정의합니다. 보스턴 집값 예측 문제는 회귀 문제이므로, 출력 레이어에서 활성화 함수를 사용하지 않습니다.

```python
# 모델 정의
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_dim=X_train.shape[1]),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # 출력층, 활성화 함수 없음
])

# 모델 컴파일
model.compile(optimizer='adam', loss='mean_squared_error')
```

3. 모델 훈련

모델을 훈련시킵니다. 여기서는 50 에포크 동안 훈련을 진행하며, 검증 데이터로 성능을 모니터링합니다.

```python
# 모델 훈련
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)
```

4. 모델 성능 평가

훈련이 완료된 후, 테스트 데이터셋을 사용하여 성능을 평가합니다.

```python
# 모델 평가
loss = model.evaluate(X_test, y_test)
print(f'Test Loss: {loss}')
```

5. 모델을 사용한 예측 (추론)

모델을 사용하여 새로운 데이터에 대해 예측을 수행할 수 있습니다.

```python
# 모델을 사용한 예측
y_pred = model.predict(X_test)

# 예측 결과 출력 (첫 5개)
for i in range(5):
    print(f"실제값: {y_test[i]}, 예측값: {y_pred[i][0]}")
```

6. 결과 시각화 (옵션)

훈련 과정에서의 손실 변화를 시각화하여 모델의 성능 향상 여부를 확인할 수 있습니다.

```python
import matplotlib.pyplot as plt

# 훈련 과정의 손실 그래프
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

7. 최종 전체 코드

```python
import tensorflow as tf
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# 보스턴 집값 데이터셋 로드
boston = load_boston()
X = boston.data
y = boston.target

# 데이터 분할 (훈련 데이터와 테스트 데이터)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 데이터 표준화 (스케일링)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 모델 정의
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_dim=X_train.shape[1]),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # 출력층, 활성화 함수 없음
])

# 모델 컴파일
model.compile(optimizer='adam', loss='mean_squared_error')

# 모델 훈련
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)

# 모델 평가
loss = model.evaluate(X_test, y_test)
print(f'Test Loss: {loss}')

# 모델을 사용한 예측
y_pred = model.predict(X_test)

# 예측 결과 출력 (첫 5개)
for i in range(5):
    print(f"실제값: {y_test[i]}, 예측값: {y_pred[i][0]}")

# 훈련 과정의 손실 그래프
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

주요 포인트:

1. **데이터 전처리**: `**StandardScaler**`를 사용해 데이터를 표준화하여 모델 학습 성능을 높였습니다.
2. **모델 정의**: `**Sequential**` 모델로 은닉층을 두 개 추가하고, 활성화 함수로 `**ReLU**`를 사용했습니다. 출력층에서는 회귀 문제이므로 활성화 함수를 사용하지 않았습니다.
3. **모델 평가**: `**evaluate()**` 함수를 사용해 테스트 데이터에서 손실값을 계산하고 성능을 평가했습니다.

4. **추론**: `predict()` 함수를 사용해 모델의 예측값을 얻었습니다.

이 코드를 실행하면 보스턴 집값 데이터를 예측하는 텐서플로우 모델을 훈련시키고 성능을 평가할 수 있습니다.

새로운 데이터 넣고 추론

새로운 데이터를 넣고 추론하는 방법을 살펴보겠습니다. 이미 훈련된 모델을 사용하여 새로운 데이터에 대해 예측을 진행하려면, `predict()` 메서드를 사용하면 됩니다. 여기서는 새로운 데이터를 모델에 넣고 예측값을 출력하는 방법을 예시로 보여드리겠습니다.

1. 새로운 데이터 입력

새로운 데이터는 `X_new`와 같이 배열로 입력됩니다. 보스턴 집값 예측 모델은 13개의 특성(features)을 받기 때문에, 새로운 데이터도 13개의 값을 가져야 합니다.

예를 들어, 다음과 같이 새로운 데이터를 준비했다고 가정합니다:

```python
# 새로운 데이터 예시 (13개의 특성값)
X_new = np.array([[0.03, 0.02, 6.1, 0.5, 0.3, 5.2, 70.0, 4.0, 2.2, 350.0, 18.4, 392.0, 4.
0]])
```

이 예시는 각 특성의 값을 배열로 넣은 것입니다. 각 특성은 보스턴 데이터셋에서의 각 열에 해당하는 값들입니다.

2. 데이터 표준화 (스케일링)

훈련 데이터에서 스케일링을 사용했으므로, 새로운 데이터도 동일한 방식으로 표준화해야 합니다. 이미 훈련에 사용한 `scaler`를 이용하여 새로운 데이터도 표준화합니다.

```python
# 새로운 데이터 표준화
X_new_scaled = scaler.transform(X_new)
```

3. 예측 수행

모델이 훈련된 후, `predict()` 메서드를 사용하여 예측을 수행합니다.

```python
# 예측 수행
y_new_pred = model.predict(X_new_scaled)
```

```
# 예측된 값 출력
print(f'새로운 데이터 예측값: {y_new_pred[0][0]}')
```

이제 전체적인 흐름을 코드로 완성하겠습니다.

전체 코드 예시 (새로운 데이터 예측)

```python
import tensorflow as tf
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# 보스턴 집값 데이터셋 로드
boston = load_boston()
X = boston.data
y = boston.target

# 데이터 분할 (훈련 데이터와 테스트 데이터)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 데이터 표준화 (스케일링)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 모델 정의
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_dim=X_train.shape[1]),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # 출력층, 활성화 함수 없음
])

# 모델 컴파일
model.compile(optimizer='adam', loss='mean_squared_error')

# 모델 훈련
model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)

# 새로운 데이터 예시 (13개의 특성값)
X_new = np.array([[0.03, 0.02, 6.1, 0.5, 0.3, 5.2, 70.0, 4.0, 2.2, 350.0, 18.4, 392.0, 4.
0]])

# 새로운 데이터 표준화
X_new_scaled = scaler.transform(X_new)

# 예측 수행
y_new_pred = model.predict(X_new_scaled)

# 예측된 값 출력
print(f'새로운 데이터 예측값: {y_new_pred[0][0]}')
```

결과:

`**y_new_pred[0][0]**`에는 새로운 데이터에 대한 예측된 집값이 출력됩니다.

주의사항:

- 새로운 데이터는 훈련 데이터와 동일한 형식과 스케일을 따라야 하므로, 반드시 `scaler`를 사용해 표준화 과정을 거쳐야 합니다.
- 새로운 데이터가 훈련 데이터와 동일한 13개의 특성을 가지고 있어야 하며, 순서가 맞아야 합니다.

모델 저장후 추론시, 스케일링에 필요한 내용은 피클로 저장하고 불러와서 사용

모델과 스케일링 정보를 각각 별도로 저장하고 불러와서 추론하는 방법을 설명하겠습니다. 이 예제에서는 **모델**과 **스케일러**를 `pickle`을 사용해 저장하고, 추론 시 이를 불러와서 사용할 것입니다.

1. 모델 저장하기

모델을 저장하기 위해 `TensorFlow`에서 제공하는 `model.save()` 메서드를 사용할 수 있습니다. 하지만, `pickle`을 사용하여 저장하고 불러오는 방법을 다룰 것이므로, 모델 저장은 `tensorflow.keras.models`에서 제공하는 `save()`와 `load_model()` 메서드를 사용하고, `scaler`는 `pickle`을 사용해 저장합니다.

2. 스케일러 저장하기

`sklearn.preprocessing.StandardScaler` 객체는 `pickle`을 사용하여 저장할 수 있습니다.

3. 모델과 스케일러 저장 및 불러오기

모델 저장

모델을 파일로 저장하기 위해서는 `TensorFlow`의 `model.save()`를 사용합니다.

```python
# 모델 저장
model.save('boston_house_price_model.h5')
```

스케일러 저장

`pickle`을 사용하여 `StandardScaler`를 저장할 수 있습니다.

```python
import pickle

# 스케일러 저장
with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
```

4. 모델과 스케일러 불러오기

모델과 스케일러를 추론 시 불러와서 사용합니다.

모델 불러오기

모델을 불러오는 방법은 `**tensorflow.keras.models.load_model()**`을 사용합니다.

```python
# 모델 불러오기
from tensorflow.keras.models import load_model

model = load_model('boston_house_price_model.h5')
```

스케일러 불러오기

`**pickle**`을 사용하여 스케일러를 불러옵니다.

```python
# 스케일러 불러오기
with open('scaler.pkl', 'rb') as f:
    scaler = pickle.load(f)
```

5. 새로운 데이터로 예측하기

이제 모델과 스케일러를 불러와 새로운 데이터에 대해 예측을 수행할 수 있습니다.

```python
# 새로운 데이터 예시 (13개의 특성값)
X_new = np.array([[0.03, 0.02, 6.1, 0.5, 0.3, 5.2, 70.0, 4.0, 2.2, 350.0, 18.4, 392.0, 4.
0]])

# 스케일러를 사용하여 데이터 표준화
X_new_scaled = scaler.transform(X_new)

# 모델을 사용한 예측
y_new_pred = model.predict(X_new_scaled)

# 예측된 값 출력
print(f'새로운 데이터 예측값: {y_new_pred[0][0]}')
```

6. 전체 코드 예시

여기서는 모델 저장, 스케일러 저장, 모델과 스케일러 불러오기, 그리고 새로운 데이터에 대한 예측을 전체 코드로 제공하겠습니다.

모델 훈련 및 저장

```python
import tensorflow as tf
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np
import pickle

# 보스턴 집값 데이터셋 로드
boston = load_boston()
X = boston.data
y = boston.target

# 데이터 분할 (훈련 데이터와 테스트 데이터)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 데이터 표준화 (스케일링)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 모델 정의
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_dim=X_train.shape[1]),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # 출력층, 활성화 함수 없음
])

# 모델 컴파일
model.compile(optimizer='adam', loss='mean_squared_error')

# 모델 훈련
model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)

# 모델 저장
model.save('boston_house_price_model.h5')

# 스케일러 저장
with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
```

모델과 스케일러 불러오기 및 새로운 데이터로 예측

```python
import tensorflow as tf
import numpy as np
import pickle
from tensorflow.keras.models import load_model

# 모델 불러오기
model = load_model('boston_house_price_model.h5')

# 스케일러 불러오기
with open('scaler.pkl', 'rb') as f:
    scaler = pickle.load(f)

# 새로운 데이터 예시 (13개의 특성값)
X_new = np.array([[0.03, 0.02, 6.1, 0.5, 0.3, 5.2, 70.0, 4.0, 2.2, 350.0, 18.4, 392.0, 4.
0]])

# 스케일러를 사용하여 데이터 표준화
X_new_scaled = scaler.transform(X_new)
```

```
# 모델을 사용한 예측
y_new_pred = model.predict(X_new_scaled)

# 예측된 값 출력
print(f'새로운 데이터 예측값: {y_new_pred[0][0]}')
```

결과:

`y_new_pred[0][0]`에는 새로운 데이터에 대한 예측값이 출력됩니다.

주의사항:

- **모델 저장**: `model.save()`를 통해 모델을 `.h5` 형식으로 저장하고, `load_model()`으로 불러옵니다.
- **스케일러 저장**: `StandardScaler`는 `pickle`을 사용하여 저장하고 불러옵니다. 데이터가 스케일링된 형태로 모델에 입력되어야 하므로, 훈련에 사용된 스케일러를 추론 시에도 반드시 사용해야 합니다.
- **데이터의 형식**: 새로운 데이터는 모델이 학습한 데이터와 동일한 형식(13개의 특성)을 가져야 하며, 순서도 일치해야 합니다.

사이킷런 분류모델도, 모델제작, 모델 제작후 튜닝, 비교 그래프 제작, 모델 성능평가

사이킷런을 사용한 분류 모델을 제작하고, 모델 튜닝 및 성능 평가를 진행하는 전체 과정을 설명하겠습니다. 예시로 **Iris 데이터셋**을 사용하여, 다양한 분류 모델을 만들고 성능을 평가하며, 튜닝을 수행한 후 모델을 비교하는 그래프를 그려보겠습니다.

1. 데이터 로드 및 전처리

사이킷런의 `load_iris()`를 사용하여 Iris 데이터셋을 불러옵니다. 그 후, 데이터를 훈련용과 테스트용으로 분할하고, 필요한 경우 전처리 작업을 수행합니다.

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Iris 데이터셋 로드
iris = load_iris()
X = iris.data
y = iris.target

# 훈련 데이터와 테스트 데이터로 분할 (80% 훈련, 20% 테스트)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 데이터 표준화 (스케일링)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. 모델 제작

여러 분류 모델을 사용하여 학습을 진행합니다. 예를 들어, **로지스틱 회귀**, **KNN**, **랜덤 포레스트**, **서포트 벡터 머신(SVM)** 등을 사용해 보겠습니다.

```python
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

# 모델들 정의
models = {
    'Logistic Regression': LogisticRegression(),
    'KNN': KNeighborsClassifier(),
    'Random Forest': RandomForestClassifier(),
    'SVM': SVC()
}
```

3. 모델 훈련 및 성능 평가

각 모델을 훈련시키고, 테스트 데이터에서 성능을 평가합니다. 성능은 `accuracy`로 평가합니다.

```python
from sklearn.metrics import accuracy_score

# 모델 훈련 및 평가
results = {}
for name, model in models.items():
    # 모델 훈련
    model.fit(X_train, y_train)
    # 예측
    y_pred = model.predict(X_test)
    # 정확도 평가
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy

# 성능 출력
for name, accuracy in results.items():
    print(f'{name}: Accuracy = {accuracy:.4f}')
```

4. 모델 튜닝 (하이퍼파라미터 튜닝)

각 모델에 대해 하이퍼파라미터 튜닝을 진행합니다. 이를 위해 **그리드 서치(Grid Search)**를 사용하여 최적의 하이퍼파라미터를 찾습니다.

```python
from sklearn.model_selection import GridSearchCV

# 하이퍼파라미터 튜닝을 위한 그리드 서치 설정
param_grid = {
    'Logistic Regression': {'C': [0.1, 1, 10]},
```

```
    'KNN': {'n_neighbors': [3, 5, 7, 9]},
    'Random Forest': {'n_estimators': [50, 100, 150]},
    'SVM': {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']}
}

tuned_results = {}

for name, model in models.items():
    print(f"Tuning {name}...")

    # GridSearchCV를 사용한 하이퍼파라미터 튜닝
    grid_search = GridSearchCV(model, param_grid[name], cv=5, n_jobs=-1)
    grid_search.fit(X_train, y_train)

    # 최적의 하이퍼파라미터 및 성능
    best_params = grid_search.best_params_
    best_score = grid_search.best_score_
    tuned_results[name] = best_score

    print(f"Best Parameters: {best_params}")
    print(f"Best Cross-Validation Score: {best_score:.4f}")

# 튜닝 후 결과 출력
for name, score in tuned_results.items():
    print(f'{name}: Best CV Score = {score:.4f}')
```

5. 모델 비교 그래프 제작

각 모델의 성능을 비교하는 그래프를 그리기 위해 `**matplotlib**`을 사용합니다.

```
python


import matplotlib.pyplot as plt

# 모델 성능 비교 그래프
models_combined = list(results.keys()) + list(tuned_results.keys())
accuracies = list(results.values()) + list(tuned_results.values())

plt.figure(figsize=(10, 6))
plt.bar(models_combined, accuracies, color='skyblue')
plt.title('Model Comparison (Before and After Tuning)', fontsize=14)
plt.xlabel('Model', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.xticks(rotation=45)
plt.show()
```

6. 모델 성능 평가

모델의 성능을 `**accuracy**` 외에 **혼동 행렬**(Confusion Matrix)과 **정밀도, 재현율, F1-score**를 포함한 평가 지표를 사용하여 평가할 수 있습니다.

```
python


from sklearn.metrics import confusion_matrix, classification_report

# 모델 성능 평가 (혼동 행렬, 분류 보고서)
for name, model in models.items():
    # 예측
    y_pred = model.predict(X_test)
```

```
    # 혼동 행렬
    cm = confusion_matrix(y_test, y_pred)
    print(f"{name} Confusion Matrix:\n{cm}")

    # 분류 보고서
    report = classification_report(y_test, y_pred)
    print(f"{name} Classification Report:\n{report}")
```

전체 코드 예시

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.model_selection import GridSearchCV
import matplotlib.pyplot as plt

# 데이터 로드 및 전처리
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 모델 정의
models = {
    'Logistic Regression': LogisticRegression(),
    'KNN': KNeighborsClassifier(),
    'Random Forest': RandomForestClassifier(),
    'SVM': SVC()
}

# 모델 훈련 및 성능 평가
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy

# 모델 성능 출력
for name, accuracy in results.items():
    print(f'{name}: Accuracy = {accuracy:.4f}')

# 하이퍼파라미터 튜닝
param_grid = {
    'Logistic Regression': {'C': [0.1, 1, 10]},
    'KNN': {'n_neighbors': [3, 5, 7, 9]},
    'Random Forest': {'n_estimators': [50, 100, 150]},
    'SVM': {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']}
}

tuned_results = {}
for name, model in models.items():
    grid_search = GridSearchCV(model, param_grid[name], cv=5, n_jobs=-1)
    grid_search.fit(X_train, y_train)
    best_params = grid_search.best_params_
    best_score = grid_search.best_score_
    tuned_results[name] = best_score
```

```
    print(f"{name}: Best Params: {best_params}, Best Score: {best_score:.4f}")

# 모델 성능 비교 그래프
models_combined = list(results.keys()) + list(tuned_results.keys())
accuracies = list(results.values()) + list(tuned_results.values())

plt.figure(figsize=(10, 6))
plt.bar(models_combined, accuracies, color='skyblue')
plt.title('Model Comparison (Before and After Tuning)', fontsize=14)
plt.xlabel('Model', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.xticks(rotation=45)
plt.show()

# 모델 성능 평가 (혼동 행렬, 분류 보고서)
for name, model in models.items():
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    print(f"{name} Confusion Matrix:\n{cm}")
    report = classification_report(y_test, y_pred)
    print(f"{name} Classification Report:\n{report}")
```

설명:

1. **모델 훈련 및 성능 평가**: 여러 분류 모델을 훈련시키고, `accuracy`를 기반으로 성능을 평가합니다.
2. **하이퍼파라미터 튜닝**: `GridSearchCV`를 사용해 각 모델의 하이퍼파라미터를 튜닝하고, 최적의 매개변수를 찾습니다.
3. **모델 비교**: 튜닝 전후 모델 성능을 바 그래프로 비교합니다.
4. **모델 성능 평가**: 혼동 행렬과 분류 보고서를 통해 모델의 성능을 자세히 분석합니다.

이 코드를 실행하면 각 모델의 성능을 시각화하고, 최적의 모델을 선택할 수 있습니다.